

Задания к работе №4 по прикладному программированию.

Все задания выполняются на языке программирования C (стандарт C99 и выше).

Реализованные приложения не должны завершаться аварийно.

Все ошибки, связанные с операциями открытия файла, должны быть обработаны; все открытые файлы должны быть закрыты.

Во всех заданиях запрещено использование глобальных переменных.

Во всех заданиях сравнение (на предмет эквивалентности или отношения порядка) вещественных чисел на уровне функции должно использовать значение эпсилон, которое (предпочтительнее) является параметром этой функции, либо (на худой конец) является значением, определённым глобально.

Во всех заданиях при пользовательском вводе с консоли необходимо организовать приглашение пользователя ко вводу в соответствии с типом и бизнес-логикой запрашиваемых данных.

Для всех реализованных алгоритмов необходимо уметь объяснить их асимптотические сложности.

Во всех заданиях при реализации функций необходимо максимально ограничивать возможность модификации (если она не подразумевается) передаваемых в функцию параметров (используйте ключевое слово *const*).

Во всех заданиях все параметры функций и вводимые (с консоли, файла, командной строки) пользователем данные должны подвергаться валидации в соответствии с типом валидируемых данных, если не сказано обратное; валидация должна зависеть от типа данных и логики применения этих данных для выполнения целевой подзадачи. При передаче аргументов приложению через командную строку, их количество также должно валидироваться.

Во всех заданиях при реализации необходимо разделять контексты работы с данными (поиск, сортировка, добавление/удаление, модификация и т. п.) и отправка данных в поток вывода / выгрузка данных из потока ввода.

Во всех заданиях запрещено использование глобальных переменных (включая *errno*).

Во всех заданиях запрещено использование оператора безусловного перехода (*goto*).

Во всех заданиях запрещено пользоваться функциями, позволяющими завершить выполнение приложения из произвольной точки выполнения, вне контекста исполнения функции *main* в качестве точки входа (то есть вызванной не рекурсивно).

1. На вход программе через аргументы командной строки подается путь ко входному файлу, флаг (флаг начинается с символа '-' или '/', второй символ - 'a' или 'd') и путь к выходному файлу. Во входном файле в каждой строке содержится информация о сотруднике (для этой информации определите тип структуры *Employee*): id (целое неотрицательное число), имя (непустая строка только из букв латинского алфавита), фамилия (непустая строка только из букв латинского алфавита), заработная плата (неотрицательное вещественное число). Программа должна считать записи из файла в динамический массив структур и в выходной файл вывести данные, отсортированные (с флагом '-a'/'a' - по возрастанию, с флагом '-d'/'d' - по убыванию) первично - по зарплате, далее (если зарплаты равны) - по фамилии, далее (если зарплаты и фамилии равны) - по именам, наконец, по id. Для сортировки коллекции экземпляров структур используйте стандартную функцию *qsort*, своя реализация каких-либо алгоритмов сортировки не допускается.
2. В текстовом файле находится информация о жителях (тип структуры *Citizen*) некоторого поселения: фамилия (непустая строка только из букв латинского алфавита), имя (непустая строка только из букв латинского алфавита), отчество (строка только из букв латинского алфавита; допускается пустая строка), дата рождения (в формате число, месяц, год), пол (символ 'M' - мужской; символ 'W' - женский), средний доход за месяц (неотрицательное вещественное число). Напишите программу, которая считывает эту информацию из файла в односвязный упорядоченный список (в порядке увеличения возраста). Информация о каждом жителе должна храниться в объекте структуры типа *Citizen*. Реализуйте возможности поиска жителя с заданными параметрами, изменение существующего жителя списка, удаления/добавления информации о жителях и возможность выгрузки данных из списка в файл (путь к файлу запрашивайте у пользователя с консоли). Добавьте возможность отменить последние $N/2$ введенных модификаций (аналог команды *Undo*); N - общее количество модификаций на текущий момент времени с момента чтения файла/последней отмены введенных модификаций.

3. Опишите тип структуры *MemoryCell*, содержащей имя переменной и её целочисленное значение. Через аргументы командной строки в программу подается файл с инструкциями вида

```
myvar=15;  
bg=25;  
ccc=bg+11;  
print ccc;  
myvar=ccc;  
bg=ccc*myvar;  
print;
```

Файл не содержит ошибок и все инструкции корректны. В рамках приложения реализуйте чтение данных из файла и выполнение всех простых арифметических операций (сложение, вычитание, умножение, целочисленное деление, взятие остатка от деления), инициализации переменной и присваивания значения переменной (=) и операции print (вывод в стандартный поток вывода либо значения переменной, имя которой является параметром операции print, либо значений всех объявленных на текущий момент выполнения переменных с указанием их имён). В каждой инструкции может присутствовать только одна из вышеописанных операций; аргументами инструкций могут выступать значение переменной, подаваемое в виде имени этой переменной, а также целочисленные константы, записанные в системе счисления с основанием 10. При инициализации переменной по необходимости требуется довыделить память в динамическом массиве структур типа *MemoryCell*; инициализация переменной (по отношению к операции перевыделения памяти) должна выполняться за амортизированную константу. Для поиска переменной в массиве используйте алгоритм дихотомического поиска, для этого ваш массив в произвольный момент времени должен находиться в отсортированном состоянии по ключу имени переменной; для сортировки массива используйте стандартную функцию *qsort*, реализовывать непосредственно какие-либо алгоритмы сортировки запрещается. Имя переменной может иметь произвольную длину и содержать только символы латинских букв, прописные и строчные буквы при этом не отождествляются. В случае использования в вычислениях не объявленной переменной необходимо остановить работу интерпретатора и вывести сообщение об ошибке в стандартный поток вывода. Предоставьте текстовый файл с инструкциями для реализованного интерпретатора и продемонстрируйте его работу. Обработайте ошибки времени выполнения инструкций.

4. Реализуйте приложение, получающее через аргументы командной строки три значения: строковые представления натуральных чисел N и k , $k < N$; а также флаг ($-f$ или $-b$). В рамках приложения необходимо:

1. Построить двусвязный кольцевой список, содержащий значения всех натуральных чисел из диапазона $[1..N]$;
2. Запустить моделирование задачи Иосифа Флавия: на каждом шаге (начиная с элемента со значением 1) удалять каждый k -й элемент кольцевого списка, проходя по списку от начала к концу (флаг $-f$) либо от конца к началу (флаг $-b$), до тех пор пока в списке не останется единственный элемент. Значение каждого удалённого элемента необходимо вывести в стандартный поток вывода. Для значения оставшегося элемента со значением x необходимо найти произведение P всех простых чисел, меньших x и вывести значение P в стандартный поток вывода.

5. На основе односвязного списка реализуйте тип многочлена от одной переменной (коэффициенты многочлена являются целыми числами). Реализуйте функции, репрезентирующие операции сложения многочленов, вычитания многочлена из многочлена, умножения многочленов, целочисленного деления многочлена на многочлен, поиска остатка от деления многочлен на многочлен, вычисления многочлена в заданной точке, нахождения производной многочлена, композиции многочленов.

Для демонстрации работы вашей программы реализуйте возможность обработки текстового файла (с чувствительностью к регистру) следующего вида:

Add($2x^2-x+2$, $-x^2+3x-1$); % сложить заданные многочлены;

Div(x^5 , x^2-1); % разделить нацело заданные многочлены.

Возможные инструкции в файле: однострочный (начинается с символа '%') комментарий, многострочный (начинается с символа '[', заканчивается символом ']', вложенность запрещена) комментарий; Add – сложение многочленов, Sub - вычитание многочлена из многочлена, Mult – умножение многочленов, Div – целочисленное деление многочлена на многочлен, Mod – остаток от деления многочлена на многочлен, Eval – вычисление многочлена в заданной точке, Diff – дифференцирование многочлена, Cmps – композиция двух многочленов.

Если в некоторой инструкции файла количество параметров меньше требуемого на 1, то это означает, что вместо первого параметра используется текущее значение сумматора. Например:

Mult(x^2+3x-1 , $2x+x^3$); % умножить два многочлена друг на друга результат сохранить в сумматор и вывести на экран;

Add($4x-8$); % в качестве первого аргумента будет взято значение из сумматора, результат будет занесен в сумматор;

Eval(1); % необходимо будет взять многочлен из сумматора и для него вычислить значение в 1.

Значение сумматора в момент начала выполнения программы равно 0.

6. На вход приложению через аргументы командной строки подаются пути ко входным текстовым файлам, содержащим выражения (в каждой строке находится одно выражение), а также флаг (--calculate или --table).

1. Флаг --calculate, программа вычисления значений арифметических выражений. Выражения в файлах могут быть произвольной структуры: содержать произвольное количество арифметических операций (сложение, вычитание, умножение, целочисленное деление, взятие остатка от деления, возведение в целую неотрицательную степень), круглых скобок (задают приоритет вычисления подвыражений).

В вашем приложении необходимо для каждого выражения из каждого входного файла:

- проверить баланс скобок для каждого выражения;
- построить обратную польскую запись для каждого выражения;
- вычислить значение выражения с использованием алгоритма вычисления выражения, записанного в обратной польской записи.

В результате работы приложения для каждого входного файла в стандартный поток вывода необходимо вывести путь к файлу и список выражений из него, для каждого выражения

вывести: исходное выражение, обратную польскую запись для выражения, значение выражения (при возможности его вычислить).

В случае обнаружения ошибки в расстановке скобок либо невозможности вычислить значение выражения, для каждого файла, где обнаружены вышеописанные ситуации, необходимо создать текстовый файл, в который выписать для каждого ошибочного выражения из исходного файла: само выражение, его порядковый номер в файле (индексация с 0) и причину невозможности вычисления.

Для решения задачи используйте собственную реализацию структуры данных вида стек на базе структуры данных вида односвязный список.

Предоставьте текстовый файл с выражениями для реализованного приложения и продемонстрируйте его работу. Обработайте ошибки времени выполнения инструкций.

2. Флаг `--table` — приложение, которое по заданной булевой формуле строит таблицу истинности.

Каждый файл содержит произвольное количество строк, в которых записаны булевы формулы. В этой формуле могут присутствовать:

- односимвольные имена логических переменных;
- константы 0 (ложь) и 1 (истина);
- `&` - оператор логической конъюнкции;
- `|` - оператор логической дизъюнкции;
- `~` - оператор логической инверсии;
- `->` - оператор логической импликации;
- `+>` - оператор логической коимпликации;
- `<>` - оператор логического сложения по модулю 2;
- `=` - оператор логической эквиваленции;
- `!` - оператор логического штриха Шеффера;
- `?` - оператор логической функции Вебба;
- операторы круглых скобок, задающие приоритет вычисления подвыражений.

Вложенность скобок произвольна. Для вычисления булевой формулы постройте бинарное дерево выражения и вычисление значения булевой формулы на конкретном наборе переменных выполняйте с помощью этого дерева. Приоритеты операторов (от высшего к низшему): 3(`~`), 2(`?`,`!`,`+>`,`&`), 1(`|`, `->`, `<>`, `=`).

В результате работы приложения необходимо получить выходной файл (имя файла псевдослучайно составляется из букв латинского алфавита и символов арабских цифр, файл должен быть размещён в одном каталоге с исходным файлом), в котором для каждого выражения должна быть построена своя таблица истинности, заданной входной булевой формулой.